Secure Shift

Guard App Onboarding Guide

Local setup · Guard App on Expo Go · Backend API with Swagger

For new contributors · Version 1.0

1. Project Overview

SecureShift is a security-guard staffing platform. The repository holds three applications plus a database, wired together so guards, employers, and admins share one backend API.

Component	What it is	Local URL
app-backend	Node.js + Express 5 REST API with MongoDB (Mongoose), JWT + 2FA auth, and Swagger docs.	http://localhost:5000
app-frontend/employer-panel	React web app (employer/admin panel).	http://localhost:3000
guard_app	React Native + Expo (TypeScript) mobile app for guards. Runs in Expo Go.	Expo dev server :8081
MongoDB	Database (runs via Docker, image mongo:6).	localhost:27017

The backend serves all routes under `/api/v1` and exposes interactive Swagger docs at `/api-docs`. Both frontends talk to it through a centralized Axios instance that attaches the JWT automatically.

2. Prerequisites

- **Node.js** v18+ and npm.
- **Git**.
- **Docker + Docker Compose** (easiest way to run the backend, DB, and employer panel together).
- **Expo Go** app installed on your physical iOS/Android phone (from the App Store / Play Store).
- Optional: **Android Studio** (Android emulator) or **Xcode** (iOS simulator, macOS only).
- Optional: **MongoDB Compass** to inspect the database.

3. Clone the repository

```
git clone https://github.com/Gopher-Industries/SecureShift.git
cd SecureShift
```

4. Run the backend + database

You can run the backend either with Docker (recommended) or directly with Node. Either way it must be running before the Guard App can log in.

4.1 Create the backend .env

Create `app-backend/.env` (never commit it).

.env file will be shared in the group by the leader

A template is available at `app-backend/.env.example` — copy it (`cp .env.example .env`) and fill in the shared values.

4.2 Option A — Docker Compose (recommended)

From the repo root, this builds and starts the backend, MongoDB, and the employer panel:

```
docker compose up --build -d
```

Verify the stack is up:

Service	URL
Backend API	http://localhost:5000
Swagger docs	http://localhost:5000/api-docs
Employer panel	http://localhost:3000
MongoDB	localhost:27017

Stop everything with `docker compose down` (add `-v` to also wipe the DB volume).

Step-by-step walkthrough (from scratch)

A quick end-to-end run. Copy the commands as you go:

1. In the backend application folder, create a new `.env` file and copy the configuration into it (see section 4.1).
2. Go back to the root of the project. The `docker-compose.yml` file is the "recipe" — it defines the containers, ports, settings, and dependencies the project is built from.
3. Make sure Docker is installed and running. Test it with the command below — if Docker responds, you're good (a fresh machine shows no containers yet).

```
docker ps
```

4. From the project root, build and start everything. The first build takes a while — let it finish.

```
docker compose up
```

5. Once it's built, run `docker ps` again to confirm the containers are up and healthy.

```
docker ps
```

6. Open the Swagger UI in your browser:

```
http://localhost:5000/api-docs
```

7. Open the employer panel. If both load, the launch succeeded.

```
http://localhost:3000
```

> **Detached vs. attached**
> Running `docker compose up` in the foreground lets you watch the build logs. Add `-d` to run it in the background once you're comfortable:

```
docker compose up --build -d
```

4.3 Option B — Run backend with Node directly

If you want hot-reload on the backend, run MongoDB (via Docker or locally) and then:

```
cd app-backend
npm install
npm run dev        # nodemon server.js (hot reload)
# or: npm start    # node server.js
```

> **Confirm it started**
> The console prints 🚀 `Server running on http://localhost:5000` and 📘 `Swagger UI: http://localhost:5000/api-docs`. Open the Swagger URL in a browser to confirm.

5. Run the Guard App in Expo Go

The Guard App is an Expo managed React Native app. Expo Go lets you run it on your phone with no native build.

5.1 Install dependencies

```
cd guard_app
npm install
```

5.2 Start the dev server

```
npm start            # expo start  (shows a QR code)
npm run android      # expo start --go --android (emulator/device)
npm run ios          # expo start --go --ios (simulator, macOS)
npm run web          # expo start --web
```

Run `npm start` inside `guard_app`. Metro starts, prints a QR code, and shows a command menu in the terminal:

```
› Press s │ switch to Expo Go
› Press a │ open Android
› Press i │ open iOS simulator
› Press w │ open web
› Press r │ reload app
› Press ? │ show all commands
```

Step 1 — Switch to Expo Go. If the terminal says `Using development build`, press `s` to switch to Expo Go. Expo Go runs on your phone with no native build.

Step 2 — Choose how to run the app:

> **Physical phone (Expo Go):** open Expo Go and scan the QR code — Android: scan inside the Expo Go app; iOS: scan with the Camera app. Your phone and computer must be on the same Wi-Fi.
>
> **Android emulator:** press `a` (requires Android Studio).
>
> **iOS simulator (macOS only):** press `i` (requires Xcode).
>
> **Web browser:** press `w`.

The bundle builds and the app opens with live reload. Press `r` to reload, `m` to toggle the menu, or `?` to list all commands.

5.3 Connect the app to the backend

Expo Go runs the app on your phone, but the phone still has to reach the backend running on your computer. The Guard App works this out automatically, with one requirement.

Your phone and computer must be on the same Wi-Fi network. Expo detects your computer's local network address and the app connects to it on port `5000` automatically — so on a physical phone this just works, as long as both devices share the same network.

The default backend port is 5000, and it works out of the box on Windows/WSL with no extra configuration.

> ⚠️ **ONLY FOR MAC USERS**

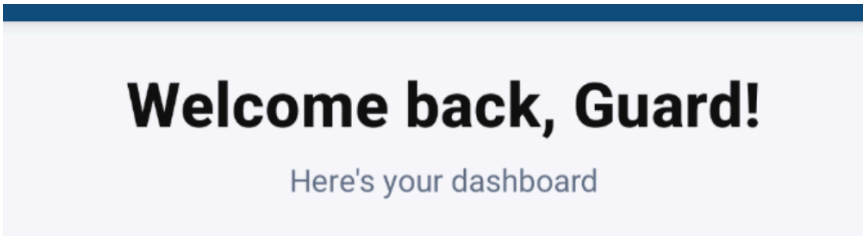
On macOS, AirPlay Receiver often occupies port 5000, so the backend is commonly run on 5001. When your backend is not on 5000, point the app at it by creating a `guard_app/.env` file:

```
EXPO_PUBLIC_API_BASE_URL=http://192.168.x.x:5001
```

Use your computer's LAN IP here, not `localhost`, when testing on a physical phone. Do not include `/api/v1` — the app appends it automatically. Restart the Expo dev server after changing `.env`.

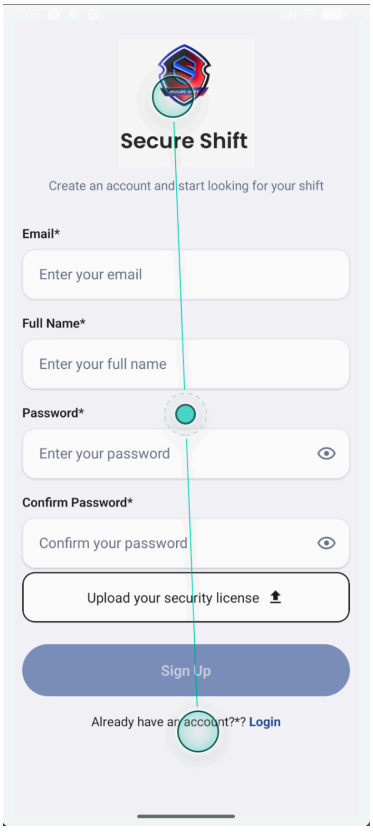
5.4 First run — sign up and get approved

On first launch the app may open straight onto the dashboard showing *"Welcome back, Guard!"* — that's a left-over session from a previous/seeded login. To create your own account, sign out first:



First launch may open a left-over "Welcome back, Guard!" session.

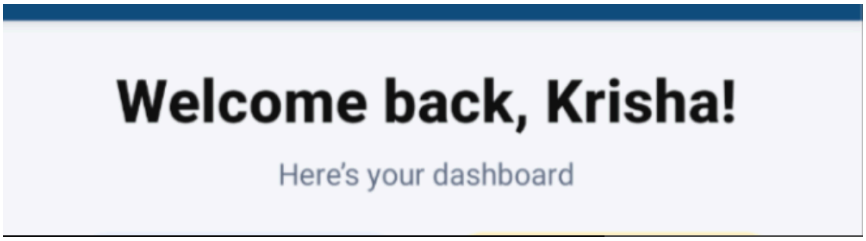
1. Open **Settings** and tap **Log out**. You'll land on the Sign Up screen.



The Secure Shift sign-up screen.

2. Fill in **Email**, **Full Name**, **Password**, and **Confirm Password**.
3. Tap **Upload your security license** and attach a photo (for testing, any image works).

4. Tap **Sign Up**.
5. Your account isn't active yet — ask your **team leader / supervisor** to approve your request. Once approved, log in and you'll reach the dashboard.



After approval, logging in with your own account shows your dashboard.

> **Already have an account?**
> On the Sign Up screen, tap **Login** at the bottom instead.

6. Repository structure

```
SecureShift/
├─ docker-compose.yml        # backend + mongo + employer panel
├─ app-backend/              # Express API
│  ├─ server.js  app.js
│  └─ src/  routes  controllers  services  models  config(swagger)  middleware
├─ app-frontend/employer-panel/   # React web (admin/employer)
└─ guard_app/                # React Native + Expo (guards)
   └─ src/  screen  components  navigation  api  lib(http)  models  theme  i18n  locales
types  utils
```

7. Dev workflow & conventions

- **Branching:** never commit to `main` directly. Create a feature branch (e.g. `frontend/yourname-payroll-page`), and open PRs into the team's integration branch — not `main`. PRs must be reviewed before merge.
- **Run tests before every PR:** `npm test` in each package you touched (`app-backend`, `guard_app`, and `app-frontend/employer-panel`). Confirm the app builds and ESLint passes.
- **Guard app scripts:** `npm run lint`, `npm run format`, `npm test` (Jest). Husky runs hooks on commit.
- **Backend scripts:** `npm run dev`, `npm run lint`, `npm run format`, `npm test` (Jest, runInBand).
- **Employer panel scripts:** `npm start`, `npm run build`, `npm test`.
- **Env files:** never commit `.env`. Copy from the `.env.example` files where provided.

You're set up — backend on :5000, Swagger at /api-docs, Guard App live in Expo Go. Happy building.